

4.6 Decorator Pattern

Decorator Design Pattern เป็นแพตเทิร์นโครงสร้างที่ช่วยเพิ่มฟังก์ชันหรือคุณสมบัติใหม่ให้กับอ็อบเจกต์โดยไม่ต้องเปลี่ยนแปลงโครงสร้างหรือโค้ดเดิมของคลาส เหมาะกับสถานการณ์ที่ต้องการเพิ่มความสามารถให้กับอ็อบเจกต์แบบ Dynamic และไม่กระทบกับอ็อบเจกต์อื่น

:: *Decorator Pattern* ::

เพิ่มฟังก์ชันหรือคุณสมบัติใหม่ให้กับอ็อบเจกต์โดยไม่ต้องเปลี่ยนแปลงโครงสร้าง
เมื่อต้องการเพิ่มความสามารถให้กับอ็อบเจกต์แบบ Dynamic และไม่กระทบกับอ็อบเจกต์อื่น

ตัวอย่างปัญหา

บ้านพื้นฐานมีคุณสมบัติมาตรฐาน แต่อาจต้องการเพิ่มคุณสมบัติพิเศษ

ตัวอย่าง Main program สร้างบ้านพร้อมระบบรักษาความปลอดภัยและเฟอร์นิเจอร์

```
House houseWithAllFeatures = new SecuritySystemDecorator(  
    new FurnitureDecorator(  
        new SmartElectricityDecorator(new BasicHouse())  
    ));
```

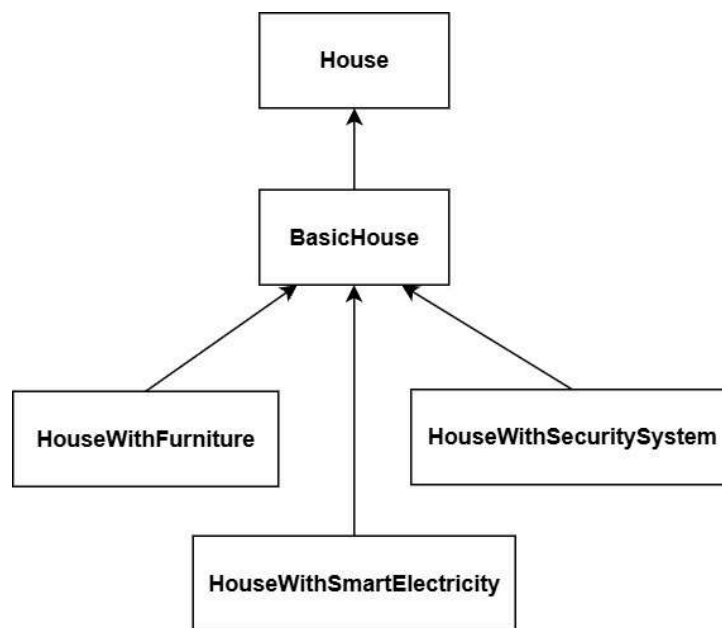
4.6.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Decorator Design Pattern

- การเพิ่มคุณสมบัติใหม่ให้กับอ็อบเจกต์ เช่น เพิ่มคุณสมบัติให้บ้าน เช่น ตกแต่งด้วยเฟอร์นิเจอร์ สีพิเศษ หรือเพิ่มระบบไฟฟ้าอัจฉริยะ
- หลีกเลี่ยงการสร้าง Subclass ที่มากเกินไป หากใช้ Inheritance เพื่อเพิ่มคุณสมบัติ อาจนำไปสู่การสร้าง Subclass จำนวนมากและโค้ดที่ซับซ้อน
- ความยืดหยุ่นในการเพิ่มความสามารถ Decorator สามารถเพิ่มคุณสมบัติให้อ็อบเจกต์ได้ในขณะ Runtime และสามารถปรับเปลี่ยนคุณสมบัติได้อย่างอิสระ
- การแยกความรับผิดชอบ (Separation of Concerns) ทำให้ฟังก์ชันการทำงานใหม่แยกออกจากโค้ดหลักของคลาสเดิม

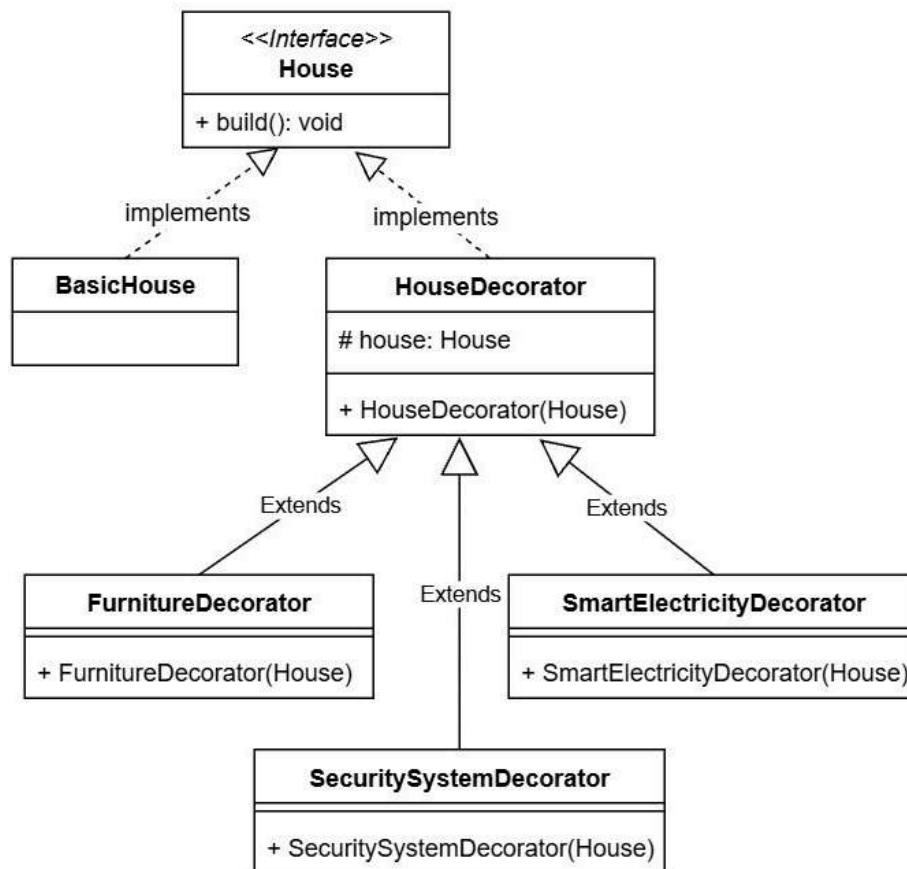
ตัวอย่างการใช้งาน

สมมติว่ากำลังพัฒนาแอปพลิเคชันที่จัดการบ้าน บ้านพื้นฐานมีคุณสมบัติมาตรฐาน เช่น โครงสร้างบ้าน อาจต้องการเพิ่มคุณสมบัติพิเศษ เช่น การตกแต่งด้วยเฟอร์นิเจอร์ การเพิ่มระบบไฟฟ้าอัจฉริยะ หรือการเพิ่มระบบรักษาความปลอดภัย หรือเพิ่มทุกอย่างทั้งเฟอร์นิเจอร์ ระบบไฟฟ้าอัจฉริยะ และระบบรักษาความปลอดภัย



ภาพที่ xx แนวคิดของ Decorator Pattern

สามารถออกแบบคลาสไดอะแกรมจากแนวคิดข้างต้นได้ดังนี้



ภาพที่ xx Decorator Pattern

Interface House กำหนดโครงสร้างพื้นฐานของบ้านที่สามารถเพิ่มคุณสมบัติได้

```

1. // Step 1: สร้าง Component Interface
2. interface House {
3.     void build();
4. }

```

Concrete Component (BasicHouse) เป็นคลาสพื้นฐานของบ้านที่มีคุณสมบัติพื้นฐาน

```

1. // Step 2: สร้าง Concrete Component สำหรับบ้านพื้นฐาน
2. class BasicHouse implements House {
3.     @Override
4.     public void build() {
5.         System.out.print("Basic House");
6.     }
7. }

```

Abstract Decorator (HouseDecorator) เป็นตัวกลางสำหรับการตกแต่งบ้าน โดยมีการเชื่อมโยงกับ House เพื่อให้สามารถเพิ่มคุณสมบัติได้แบบ Dynamic

```
1. // Step 3: สร้าง Abstract Decorator
2. 1. abstract class HouseDecorator implements House {
3.     protected House house;
4.
5.     public HouseDecorator(House house) {
6.         this.house = house;
7.     }
8.
9.     @Override
10.    public void build() {
11.        house.build();
12.    }
13.}
```

Concrete Decorators (FurnitureDecorator, SmartElectricityDecorator, SecuritySystemDecorator) เป็นคลาสที่เพิ่มคุณสมบัติพิเศษให้กับบ้าน เช่น เพอร์นิเจอร์ ระบบไฟฟ้าอัจฉริยะ หรือระบบรักษาความปลอดภัย

```
1. // Step 4: สร้าง Concrete Decorator สำหรับคุณสมบัติพิเศษ
2. 2. class FurnitureDecorator extends HouseDecorator {
3.
4.     public FurnitureDecorator(House house) {
5.         super(house);
6.     }
7.
8.     @Override
9.     public void build() {
10.        super.build();
11.        System.out.print(" + Furniture");
12.    }
13.}
```

```
1. 1. class SmartElectricityDecorator extends HouseDecorator {
2.
3.     public SmartElectricityDecorator(House house) {
4.         super(house);
5.     }
6.
7.     @Override
8.     public void build() {
9.        super.build();
10.       System.out.print(" + Smart Electricity System");
11.    }
12. }
```

```

1. class SecuritySystemDecorator extends HouseDecorator {
2.
3.     public SecuritySystemDecorator(House house) {
4.         super(house);
5.     }
6.
7.     @Override
8.     public void build() {
9.         super.build();
10.        System.out.print(" + Security System");
11.    }
12. }

```

การใช้งาน สามารถเพิ่มคุณสมบัติให้บ้านได้แบบ Dynamic โดยไม่ต้องแก้ไขโค้ดเดิมของ BasicHouse

```

1. // Step 5: การใช้งาน Decorator Pattern
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // บ้านพื้นฐาน
6.         House basicHouse = new BasicHouse();
7.         System.out.print("Building: ");
8.         basicHouse.build();
9.         System.out.println();
10.
11.        // บ้านพร้อมเฟอร์นิเจอร์
12.        House houseWithFurniture = new FurnitureDecorator(
13.            new BasicHouse());
14.        System.out.print("Building: ");
15.        houseWithFurniture.build();
16.        System.out.println();
17.
18.        // บ้านพร้อมระบบไฟฟ้าอัจฉริยะ
19.        House houseWithSmartElectricity = new
20.            SmartElectricityDecorator(new BasicHouse());
21.        System.out.print("Building: ");
22.        houseWithSmartElectricity.build();
23.        System.out.println();
24.
25.        // บ้านพร้อมระบบรักษาความปลอดภัยและเฟอร์นิเจอร์
26.        House houseWithAllFeatures = new SecuritySystemDecorator(
27.            new FurnitureDecorator(
28.                new SmartElectricityDecorator(new BasicHouse())
29.            )
30.        );
31.        System.out.print("Building: ");
32.        houseWithAllFeatures.build();
33.        System.out.println();
34.    }
35. }

```

ผลลัพธ์

Building: Basic House

Building: Basic House + Furniture

Building: Basic House + Smart Electricity System

Building: Basic House + Smart Electricity System + Furniture + Security System

ข้อดีของ Decorator Pattern

- เพิ่มคุณสมบัติใหม่ให้อ็อบเจกต์ได้แบบ Dynamic ไม่จำเป็นต้องแก้ไขโค้ดเดิมหรือสร้าง Subclass ใหม่
- ยืดหยุ่นและปรับแต่งได้ง่าย สามารถเพิ่ม Decorator หลายตัวเพื่อสร้างอ็อบเจกต์ที่มีคุณสมบัติหลากหลาย
- ส่งเสริมการออกแบบตามหลัก Single Responsibility Principle (SRP) แต่ละ Decorator มีความรับผิดชอบชัดเจนในการเพิ่มคุณสมบัติเฉพาะ

4.6.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Decorator Pattern เหมาะสำหรับการเพิ่มคุณสมบัติหรือความสามารถใหม่ให้กับอ็อบเจกต์โดยไม่ต้องเปลี่ยนแปลงโครงสร้างเดิม และรองรับการปรับแต่งคุณสมบัติได้แบบ Dynamic ตัวอย่างปัญหาที่สามารถแก้ไขได้ด้วย Decorator Pattern มีดังนี้

ตัวอย่างที่ 1 การออกแบบระบบจัดการข้อความ (Text Processing) ระบบจัดการข้อความต้องรองรับการปรับแต่งข้อความ เช่น การเข้ารหัสข้อความ (Encryption) การบีบอัดข้อความ (Compression) และการเพิ่มลายน้ำ (Watermark) การเขียนโค้ดสำหรับทุกการผสมพีเจอร์จะทำให้โค้ดซับซ้อน ใช้ Decorator Pattern เพื่อเพิ่มความสามารถให้กับข้อความ เช่น การเข้ารหัส การบีบอัด หรือการเพิ่มลายน้ำ

```
1. // Component Interface
2. interface Text {
3.     String process();
4. }
```

```
1. // Concrete Component
2. class PlainText implements Text {
3.
4.     private String content;
5.
6.     public PlainText(String content) {
```

```

7.         this.content = content;
8.     }
9.
10.    @Override
11.    public String process() {
12.        return content;
13.    }
14.}

```

```

1. // Abstract Decorator
2. abstract class TextDecorator implements Text {
3.
4.     protected Text text;
5.
6.     public TextDecorator(Text text) {
7.         this.text = text;
8.     }
9.
10.    @Override
11.    public String process() {
12.        return text.process();
13.    }
14.}

```

```

1. // Concrete Decorators
2. class EncryptionDecorator extends TextDecorator {
3.
4.     public EncryptionDecorator(Text text) {
5.         super(text);
6.     }
7.
8.     @Override
9.     public String process() {
10.        return "Encrypted(" + super.process() + ")";
11.    }
12.}

```

```

1. class CompressionDecorator extends TextDecorator {
2.
3.     public CompressionDecorator(Text text) {
4.         super(text);
5.     }
6.
7.     @Override
8.     public String process() {
9.        return "Compressed(" + super.process() + ")";
10.    }
11.}

```

```

1. class WatermarkDecorator extends TextDecorator {
2.
3.     public WatermarkDecorator(Text text) {
4.         super(text);
5.     }

```

```

6.
7.     @Override
8.     public String process() {
9.         return super.process() + " [Watermarked]";
10.    }
11.}

1. // Main Class
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // ข้อความธรรมดา
6.         Text plainText = new PlainText("Hello, World!");
7.
8.         // ข้อความเข้ารหัส
9.         Text encryptedText =
10.            new EncryptionDecorator(plainText);
11.         System.out.println(encryptedText.process());
12.
13.        // ข้อความเข้ารหัสและบีบอัด
14.        Text compressedEncryptedText = new
15.            CompressionDecorator(encryptedText);
16.        System.out.println(compressedEncryptedText.process());
17.
18.        // ข้อความพร้อมลายน้ำ
19.        Text fullDecoratedText = new
20.            WatermarkDecorator(compressedEncryptedText);
21.        System.out.println(fullDecoratedText.process());
22.    }
23.}

```

ผลลัพธ์

Encrypted(Hello, World!)

Compressed(Encrypted(Hello, World!))

Compressed(Encrypted(Hello, World!)) [Watermarked]

ตัวอย่างที่ 2 การออกแบบระบบอีคอมเมิร์ซ (E-Commerce System) ระบบอีคอมเมิร์ซต้องรองรับฟีเจอร์สำหรับคำสั่งซื้อ เช่น การเพิ่มประกันสินค้า การจัดส่งด่วน (Express Delivery) และการเพิ่มบริการห่อของขวัญ หากต้องสร้าง Subclass สำหรับทุกการผสมฟีเจอร์ จะทำให้โค้ดซับซ้อน ใช้ Decorator Pattern เพื่อเพิ่มฟีเจอร์ให้คำสั่งซื้อ เช่น การเพิ่มประกันสินค้า หรือการจัดส่งด่วน

```

1. // Step 1: สร้าง Component Interface สำหรับ Order
2. interface Order {
3.     String getDescription();
4.     double getCost();
5. }

```

```

1. // Step 2: สร้าง Concrete Component สำหรับ Order พื้นฐาน
2. class BasicOrder implements Order {
3.
4.     @Override
5.     public String getDescription() {
6.         return "Basic Order";
7.     }
8.
9.     @Override
10.    public double getCost() {
11.        return 50.0; // ราคาเริ่มต้นของคำสั่งซื้อ
12.    }
13.}

```

```

1. // Step 3: สร้าง Abstract Decorator
2. abstract class OrderDecorator implements Order {
3.
4.    protected Order order;
5.
6.    public OrderDecorator(Order order) {
7.        this.order = order;
8.    }
9.
10.    @Override
11.    public String getDescription() {
12.        return order.getDescription();
13.    }
14.
15.    @Override
16.    public double getCost() {
17.        return order.getCost();
18.    }
19.}

```

```

1. // Step 4: สร้าง Concrete Decorators สำหรับฟีเจอร์พิเศษ
2. class WarrantyDecorator extends OrderDecorator {
3.
4.    public WarrantyDecorator(Order order) {
5.        super(order);
6.    }
7.
8.    @Override
9.    public String getDescription() {
10.        return super.getDescription() + " + Warranty";
11.    }
12.
13.    @Override
14.    public double getCost() {
15.        return super.getCost() + 15.0; // เพิ่มค่าประกันสินค้า
16.    }
17.}

```

```

1. class ExpressDeliveryDecorator extends OrderDecorator {
2.
3.     public ExpressDeliveryDecorator(Order order) {
4.         super(order);
5.     }
6.
7.     @Override
8.     public String getDescription() {
9.         return super.getDescription() + " + Express Delivery";
10.    }
11.
12.    @Override
13.    public double getCost() {
14.        return super.getCost() + 20.0; // เพิ่มค่าจัดส่งด่วน
15.    }
16.}

```

```

1. class GiftWrapDecorator extends OrderDecorator {
2.
3.     public GiftWrapDecorator(Order order) {
4.         super(order);
5.     }
6.
7.     @Override
8.     public String getDescription() {
9.         return super.getDescription() + " + Gift Wrapping";
10.    }
11.
12.    @Override
13.    public double getCost() {
14.        return super.getCost() + 10.0; // เพิ่มค่าห่อของขวัญ
15.    }
16.}

```

```

1. // Step 5: การใช้งาน Decorator Pattern
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // คำสั่งซื้อพื้นฐาน
6.         Order basicOrder = new BasicOrder();
7.         System.out.println(basicOrder.getDescription()
8.             + " | Total Cost: $" + basicOrder.getCost());
9.
10.        // คำสั่งซื้อพร้อมประกันสินค้า
11.        Order orderWithWarranty =
12.            new WarrantyDecorator(basicOrder);
13.        System.out.println(orderWithWarranty.getDescription()
14.            + " | Total Cost: $"
15.            + orderWithWarranty.getCost());
16.
17.        // คำสั่งซื้อพร้อมจัดส่งด่วน
18.        Order orderWithExpressDelivery = new
19.            ExpressDeliveryDecorator(basicOrder);
20.        System.out.println(orderWithExpressDelivery
21.            .getDescription() + " | Total Cost: $"
22.            + orderWithExpressDelivery.getCost());

```

```

23.
24.      // คำสั่งซื้อพร้อมพีเจอร์ทั้งหมด
25.      Order fullOrder = new GiftWrapDecorator(
26.          new ExpressDeliveryDecorator(
27.              new WarrantyDecorator(new BasicOrder())
28.          ));
29.      System.out.println(fullOrder.getDescription()
30.          + " | Total Cost: $" + fullOrder.getCost());
31.  }
32.}

```

ผลลัพธ์

Basic Order | Total Cost: \$50.0

Basic Order + Warranty | Total Cost: \$65.0

Basic Order + Express Delivery | Total Cost: \$70.0

Basic Order + Warranty + Express Delivery + Gift Wrapping | Total Cost: \$95.0

4.7 Proxy Pattern

Proxy Pattern เป็นแพตเทิร์นโครงสร้างที่ใช้สร้างตัวแทน (Proxy) สำหรับควบคุมหรือจัดการการเข้าถึงอ็อบเจกต์จริง (Real Object) Proxy สามารถช่วยเพิ่มฟังก์ชัน เช่น การควบคุมการเข้าถึง การโหลดข้อมูลแบบ Lazy Loading หรือการทำงานร่วมกับเครือข่ายได้อย่างมีประสิทธิภาพ

:: Proxy Pattern ::

สร้างตัวแทน (Proxy) สำหรับควบคุมหรือจัดการการเข้าถึงอ็อบเจกต์จริง

ช่วยเพิ่มฟังก์ชัน เช่น การควบคุมการเข้าถึง การโหลดข้อมูลแบบ Lazy Loading

ตัวอย่างปัญหา

บ้านอัจฉริยะมีระบบล็อกประตูที่สามารถควบคุมผ่าน Proxy เพื่อเพิ่มความปลอดภัย เช่น ต้องยืนยันตัวตนก่อนที่จะเข้าถึงระบบล็อกประตูจริง

ตัวอย่าง Main program

```

// สร้าง Proxy สำหรับระบบล็อกบ้าน กำหนดชื่อผู้ได้รับอนุญาต
DoorLock doorLock = new DoorLockProxy("วาสนา");

// ผู้ใช้พยายามปลดล็อกประตู
doorLock.unlock("วาสนา");

```

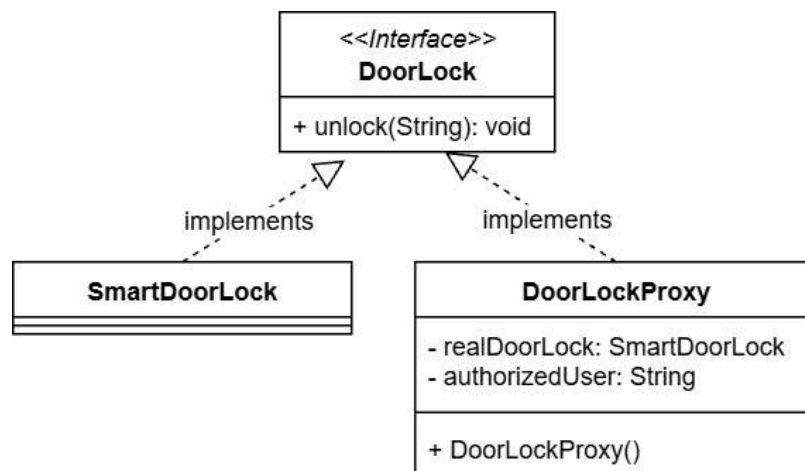
4.7.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Proxy Design Pattern

- การควบคุมการเข้าถึง (Access Control) ใช้ Proxy เพื่อควบคุมสิทธิ์การเข้าถึงอ็อบเจกต์จริง เช่น ระบบล็อกบ้านที่ต้องยืนยันตัวตนก่อนปลดล็อก
- การโหลดข้อมูลแบบ Lazy Loading Proxy ใช้แทนการโหลดอ็อบเจกต์ที่ใช้ทรัพยากรมาก เช่น โหลดข้อมูลของบ้านขนาดใหญ่เมื่อจำเป็นเท่านั้น
- การทำงานร่วมกับเครือข่าย (Remote Proxy) ใช้ Proxy เป็นตัวแทนในการเรียกใช้อ็อบเจกต์ที่อยู่บนเซิร์ฟเวอร์อื่น เช่น ควบคุมระบบบ้านผ่านเครือข่าย
- การเพิ่มฟังก์ชันเสริม (Additional Functionality) เช่น การบันทึกการใช้งาน หรือการตรวจสอบการเข้าถึง

ตัวอย่างการใช้งาน

สมมติว่าบ้านอัจฉริยะมีระบบล็อกประตูที่สามารถควบคุมผ่าน Proxy ได้ เพื่อเพิ่มความปลอดภัย เช่น ต้องยืนยันตัวตนก่อนที่จะเข้าถึงระบบล็อกประตูจริง



ภาพที่ xx Proxy Pattern สำหรับระบบล็อกประตู

DoorLock (Interface) กำหนดโครงสร้างของระบบล็อก เช่น เมธอด unlock

```
1. // Step 1: สร้าง Interface
2. interface DoorLock {
3.     void unlock(String user);
4. }
```

SmartDoorLock (Real Object) เป็นระบบล็อกจริงที่ดำเนินการปลดล็อก

```
1. // Step 2: สร้าง Real Object
2. class SmartDoorLock implements DoorLock {
3.     @Override
4.     public void unlock(String user) {
5.         System.out.println(user + " ได้เปิดประตู.");
6.     }
7. }
```

DoorLockProxy (Proxy) ทำหน้าที่ตรวจสอบสิทธิ์การเข้าถึงก่อนที่จะเรียกใช้ SmartDoorLock

```
1. // Step 3: สร้าง Proxy
2. class DoorLockProxy implements DoorLock {
3.
4.     private SmartDoorLock realDoorLock;
5.     private String authorizedUser;
6.
7.     public DoorLockProxy(String authorizedUser) {
8.         this.realDoorLock = new SmartDoorLock();
9.         this.authorizedUser = authorizedUser;
10.    }
11.
12.    @Override
13.    public void unlock(String user) {
14.        if (user.equals(authorizedUser)) {
15.            realDoorLock.unlock(user);
16.        } else {
17.            System.out.println("Access denied for user: "
18.                               + user);
19.        }
20.    }
21.}
```

Main ใช้ DoorLockProxy เพื่อควบคุมการเข้าถึง SmartDoorLock

```
1. // Step 4: การใช้งาน Proxy Pattern
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้าง Proxy สำหรับระบบล็อกบ้าน กำหนดชื่อผู้ได้รับอนุญาต
6.         DoorLock doorLock = new DoorLockProxy("วาสนา");
7.
8.         // ผู้ใช้พยายามปลดล็อกประตู
9.         doorLock.unlock("วาสนา"); // อนุญาตให้เข้าถึง
10.    }
11.}
```

ผลลัพธ์

วาสนา ได้เปิดประตู

สถานการณ์อื่นที่เหมาะสมกับ Proxy Pattern สำหรับบ้าน

- การควบคุมกล้องวงจรปิด (Security Camera) Proxy สามารถใช้ตรวจสอบสิทธิ์ก่อนให้ผู้ใช้งานเข้าถึงกล้อง
- การโหลดข้อมูลแบบ Lazy Loading สำหรับแผนผังบ้าน (Blueprints) Proxy ใช้สำหรับโหลดข้อมูลแผนผังบ้านเฉพาะเมื่อจำเป็น
- การจัดการพลังงานอัจฉริยะ (Smart Energy Management) Proxy สามารถติดตามและควบคุมการเปิด/ปิดอุปกรณ์ไฟฟ้าในบ้านได้

ประเภทของ Proxy

- Remote Proxy ใช้ในกรณีที่ออบเจกต์จริงอยู่บนเซิร์ฟเวอร์ระยะไกล เช่น ควบคุมบ้านผ่านเครือข่าย
- Virtual Proxy ใช้สำหรับการโหลดข้อมูลขนาดใหญ่แบบ Lazy Loading เช่น การแสดงแผนผังบ้านเมื่อผู้ใช้งานร้องขอ
- Protection Proxy ใช้สำหรับควบคุมการเข้าถึง เช่น การตรวจสอบสิทธิ์ก่อนเข้าถึงระบบบ้าน
- Logging Proxy ใช้บันทึกการใช้งาน เช่น การติดตามว่ามีใครเข้าถึงระบบบ้านบ้าง

ข้อดีของ Proxy Pattern

- ควบคุมการเข้าถึงได้ง่าย เพิ่มความปลอดภัย และลดความเสี่ยงจากการเข้าถึงโดยไม่ได้รับอนุญาต
- สนับสนุนการโหลดแบบ Lazy Loading ลดการใช้ทรัพยากรสำหรับออบเจกต์ที่ไม่จำเป็น
- รองรับการทำงานกับระบบเครือข่าย ใช้ Proxy สำหรับการเรียกใช้งานออบเจกต์บนเซิร์ฟเวอร์ระยะไกล
- เพิ่มฟังก์ชันเสริม เช่น การบันทึกการใช้งาน หรือการตรวจสอบการเข้าถึง

Proxy Pattern เหมาะสำหรับสถานการณ์ที่ต้องการควบคุมการเข้าถึงออบเจกต์หรือเพิ่มฟังก์ชันเสริม เช่น ระบบล็อกบ้านอัจฉริยะ การควบคุมกล้องวงจรปิด หรือการโหลดข้อมูลแบบ Lazy Loading ช่วยเพิ่มความปลอดภัยและประสิทธิภาพในการใช้งาน

4.7.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Proxy Pattern ใช้สำหรับจัดการการเข้าถึงอ็อบเจกต์จริง โดยการเพิ่มตัวแทน (Proxy) ที่สามารถควบคุมหรือเพิ่มฟังก์ชันการทำงานก่อนเข้าถึงอ็อบเจกต์จริง ตัวอย่างปัญหาในงานจริงที่สามารถแก้ไขได้ด้วย Proxy Pattern มีดังนี้

ตัวอย่างที่ 1 การควบคุมการเข้าถึงกล้องวงจรปิด (Security Camera System) ในระบบกล้องวงจรปิดของบ้าน เจ้าของบ้านต้องการให้เฉพาะผู้ที่มีสิทธิ์เท่านั้นสามารถดูภาพจากกล้องได้ สามารถใช้ Proxy เพื่อควบคุมการเข้าถึงภาพจากกล้อง โดยตรวจสอบสิทธิ์ก่อนที่จะเข้าถึงกล้องจริง

```
1. // Interface สำหรับกล้องวงจรปิด
2. interface Camera {
3.     void stream(String user);
4. }

1. // Real Object สำหรับกล้องวงจรปิด
2. class SecurityCamera implements Camera {
3.     @Override
4.     public void stream(String user) {
5.         System.out.println(user + " is streaming
6.                                 the camera feed.");
7.     }
8. }

1. // Proxy สำหรับควบคุมการเข้าถึง
2. class CameraProxy implements Camera {
3.
4.     private SecurityCamera realCamera;
5.     private String authorizedUser;
6.
7.     public CameraProxy(String authorizedUser) {
8.         this.realCamera = new SecurityCamera();
9.         this.authorizedUser = authorizedUser;
10.    }
11.
12.    @Override
13.    public void stream(String user) {
14.        if (user.equals(authorizedUser)) {
15.            realCamera.stream(user);
16.        } else {
17.            System.out.println("Access denied for user: " + user);
18.        }
19.    }
20.}

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
```

```
4.  
5.         Camera camera = new CameraProxy("วาสนา");  
6.  
7.         // ผู้ใช้พยายามสตรีมวิดีโอ  
8.         camera.stream("วาสนา"); // อนุญาต  
9.         camera.stream("กันเกรา"); // ปฏิเสธ  
10.    }  
11.}
```

ผลลัพธ์

วาสนา is streaming the camera feed.

Access denied for user: กันเกรา